PRIPAXOS: Priority-Driven Geo-Distributed State Machine Replication with Near-Optimal WAN Delays

OSDI'26 Submission #1855

Abstract

This paper introduces PRIPAXOS, a novel consensus protocol designed for geo-distributed state machine replication. Unlike existing protocols, which resolve conflicting dependencies through inter-replica coordination, incurring a minimum of three wide-area message delays, PRIPAXOS reduces this requirement to strictly fewer than three. The key innovation in PRIPAXOS is a priority-based scheduling mechanism: by statically assigning priorities to replicas, each node can independently and deterministically establish a globally consistent command order. Given the theoretical impossibility of achieving crash-tolerant consensus on conflicting commands within two message delays, PRIPAXOS approaches the fundamental optimal bound on latency. Extensive experiments demonstrate that compared to the state-of-the-art (SwiftPaxos), PRIPAXOS reduces average latency by 13.6% and achieves up to 4.4× higher throughput.

1 Introduction

Modern cloud services (e.g., Google Spanner [9], Apache ZooKeeper [1], and AWS Dynamo [10]) usually span multiple geographically distributed data centers, necessitating the replication of critical components across these sites to ensure high availability and fault tolerance [5, 23, 42]. To maintain strong consistency (e.g., linearizability [14]) across replicas, providers employ state machine replication (SMR) [17, 18, 35, 36, 44]. In SMR, each site hosts a replica of the service, modeled as a deterministic state machine, and protocols coordinate the execution of client commands to keep all replicas synchronized. This approach guarantees that every command appears to execute atomically and instantaneously at all sites, preserving the illusion of a single, unified system state across data centers.

Ideal geo-distributed SMR protocols must satisfy two key requirements: ❶ high throughput under heavy client concurrency, and ❷ low latency when client concurrency is light [6, 33, 35, 45]. Achieving high throughput is primarily constrained by CPU and network bandwidth, making balanced load distribution and communication across replicas essential to prevent workload imbalance. In contrast, minimizing latency depends largely on the protocol's message delay.

However, existing protocols face inherent trade-offs that make it challenging to optimize both throughput and latency simultaneously (see Figure 1). Broadly, these protocols fall into two categories. Leader-centric protocols, such as those in the Paxos lineage (e.g., Fast Paxos [18], SwiftPaxos [34]), coordinate all replicas through a leader or primary to impose a total order on commands. This approach centralizes the coordination workload, often resulting in a bottleneck at the leader under heavy load. Alternatively, leaderless, majority-based designs (e.g., EPaxos [25]) eliminate a single point of coordination and instead derive command order via quorum-based agreement. While this mitigates the bottleneck, it introduces additional message delays and imposes substantial CPU overhead to compute dependencies during conflicting commands.

Despite extensive research in recent years, achieving efficient consensus with the minimal possible number of WAN message delays (equivalent to ❶ and ❷) remains unresolved. Protocols that claim to complete in two message delays, such as the Lamport method [20], either rely on restrictive system assumptions or encounter significant practical limitations, resulting in limited adoption (see §2.2). Our work formally demonstrates that, under realistic partial synchrony and crash-fault tolerance, consensus within two message delays is theoretically unattainable (see Theorem1).

Furthermore, we observe that both leader-based and leaderless protocols can be subsumed under a unified priority scheduling framework [15, 26, 27]. Specifically, leader-based protocols correspond to the special case where a single node is assigned the highest ordering priority, while leaderless protocols assign equal priority to all nodes [3, 13, 16, 32, 41, 43]. This unification leads to our central insight: by generalizing the notion of priority, it is possible to design protocols that interpolate between these two extremes and potentially achieve more favorable trade-offs.

Building upon this unified notion, we propose PRIPAXOS, a novel SMR protocol designed to deliver both high throughput and low latency in geo-replicated environments. In PRIPAXOS, each replica is assigned a unique priority before executing user commands. When a replica proposes a command, it broadcasts the proposal and processes responses from other replicas; any commands received from higher-priority replicas are deterministically ordered before its own. Since the execution of a command depends only on commands ordered

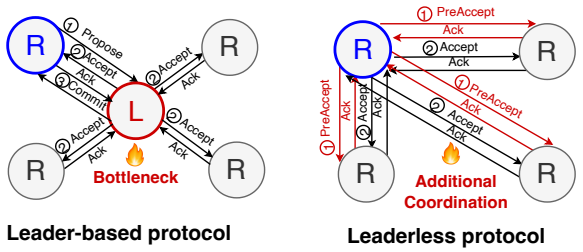

Figure 1: Leader-based protocol and leaderless protocol.

prior to it, a replica needs only await responses from higher-priority nodes to independently establish a globally consistent execution order [8,11,29]. This approach allows PRIPAXOS to execute commands in strictly fewer than three WAN message delays. Meanwhile, this new communication pattern alleviates the leader bottleneck inherent in leader-based protocols (e.g., Multi-Paxos [18]), thereby improving overall throughput.

As with other leader-based protocols, the performance of PRIPAXOS can be sensitive to the assignment of replica priorities. To address this, we introduce a novel heuristic for efficient priority allocation, which selects the high-priority node(s) to optimize PRIPAXOS 's performance (§4.4). Furthermore, PRIPAXOS supports dynamic priority re-assignment at runtime, allowing the system to adapt to changes in network conditions or workload distribution (§4.5).

PRIPAXOS is robust to crash failures. We present a priority-driven recovery mechanism in §4.3 that ensures consistency is maintained even in the presence of replica crashes. Because command ordering is determined by dependency sets rather than by specific replicas, recovery can be performed efficiently without significant overhead. Consequently, PRIPAXOS's recovery overhead is comparable to that of other majority-based consensus protocols (see Figure 10).

We implemented PRIPAXOS on top of the SwiftPaxos's codebase [34] and evaluated its performance using standard benchmarks (YCSB [2]) across a range of experimental conditions. In our experiments, PRIPAXOS is compared against two leader-based protocols (SwiftPaxos and Fastpaxos) and three leaderless protocols (EPaxos, Mencius, and Paxos). Experimental results demonstrate that, compared to the state-of-the-art SwiftPaxos, PRIPAXOS reduces average latency by 13.6% and achieves up to 4.4× higher throughput.

Contributions. The main contributions of this work are:

- We formalize an optimality theorem for consensus protocols and prove that efficiently executing conflicting commands within two message delays is impossible.
- We propose a unified priority-based framework that systematically explores the protocol design space between the two extremes of leader-based and leaderless approaches.
- We introduce PRIPAXOS, a novel consensus protocol that employs priority-based ordering to establish a unique, globally consistent command sequence, thereby eliminating the need for inter-replica dependency computation.
- We implement and evaluate PRIPAXOS across five AWS regions, and release the source code at `https://github.com/osdi261855/Pripaxos`.

The remainder of the paper is organized as follows: §2 describes the system model, desired properties, and motivation for the optimality of PRIPAXOS's design. §3 provides an overview of PRIPAXOS with illustrative examples. §4 presents the detailed design of PRIPAXOS. §5 reports experimental results. §6 concludes the paper.

2 Background and Theorem

2.1 System Model and Notations

System Model. We consider a geo-distributed, message-passing system comprising n sites interconnected via a wide-area network (WAN). Each site hosts a server process and an associated set of clients. Processes are subject to crash failures but are assumed to be non-Byzantine. Communication between clients and replicas is asynchronous. Let R denote the set of replicas, with $N = 2f + 1$ to tolerate up to f crash faults while maintaining safety [31]. The key space is global, allowing any client to access any key in the system.

When a replica receives a command from a client, it associates the command with a distinct instance. Typically, each instance contains a single command, although a replica may opt to batch multiple commands into a single instance for efficiency. Each instance is assigned a globally unique identifier, composed of the originating replica's unique ID and a monotonically increasing number local to that replica.

To achieve agreement on committed commands, the classical consensus problem requires protocols to satisfy the following three properties [18]:

Non-triviality: Any command committed by any replica must have been proposed by clients

Consistency: If any replica commits two conflicting commands x and y (i.e., access the same key concurrently), then all replicas commit them in the same order.

Liveness: If a command x is committed by some replica, then x is eventually committed by all non-faulty replicas.

2.2 Reasoning About Optimality

In geo-distributed deployments, local latency is negligible compared to inter-region delays; thus, command latency is primarily determined by inter-region communication. Consequently, our analysis focuses on inter-replica communication and omits the local client-to-replica submission step [21,37].

We now analyze the optimal performance and the minimum number of message delays required to achieve consensus. Prior work, such as Lamport's Collision-Fast Learning theorem [20], demonstrates that consensus can be reached in 2δ message delays under certain conditions. However, while correctness is maintained in the presence of partial replica failures, the 2δ fast path becomes unavailable upon failures, causing the system to revert to a slower path.

According to the Collision-Fast Learning theorem, there are two special cases that consistently enable a 2δ fast path:

Symbol	Description
δ	Single inter-region message delay
$R_1 \succ R_2$	Replica R_1 has higher priority than replica R_2
$x \rightarrow y$	Command y depends on command x
$x \rightarrow_r y$	Replica R received x before proposing y
$x \rightarrow_p y$	The order $x \rightarrow y$ decided by *Rule 2*
$x \rightsquigarrow y$	Replica R received x before y

Table 1: Summary of frequently used symbols throughout this paper.

Special Case 1 ***(SC1)****:* Given $f = 1$, where the system can tolerate at most one replica failure. In this scenario, every replica capable of executing commands can also propose a command order, and there exists at least one replica that can propose orderings but cannot execute commands.
Special Case 2: ***(SC2)*** The command proposer is always responsible for determining the command order.

Nevertheless, ***SC1*** is somewhat restrictive, as it imposes a stringent limitation on f and requires the presence of a non-executing replica. In contrast, ***SC2*** seems to be more practical, where replicas can concurrently propose, order, and execute commands. In the following, we show that **SC2** cannot be generally realized in practice. We assume that a practical consensus protocol must satisfy the three key properties(defined in §2.1): (1) Non-triviality, (2) Consistency, and (3) Liveness.

Theorem 1. Given a system that tolerates more than one fault, it is impossible for any consensus protocol to execute conflicting commands within strictly two message delays.

Specifically, we define the progress of command execution within strictly two message delays as follows:

- **Two message delays:** A command is considered to be executed in two cross-region message exchanges (2δ).
- **Non-blocking:** Each replica can execute commands without waiting for an additional signal. Prohibited blocking behaviors include waiting for locks to become available or awaiting messages from other clients or replicas. In a case, blocking all other commands to execute self-proposed commands serially trivially achieves 2δ delays, while this is against our objective of minimizing overall latency.
- **Non-abort:** In the absence of replica failures, every command completes in a single execution attempt. Similarly to the non-blocking property, eliminating retries benefits overall latency. Otherwise, blocking can always be resolved by retrying until the command proceeds without contention.

Proof. Consider a replica set $R = \{R_1, \ldots, R_n\}$, where each replica can propose commands, determine orderings, and execute commands. Suppose R_1 and R_n concurrently propose two conflicting commands, x and y, respectively. If it were possible to execute conflicting commands within 2δ, then both R_x and R_y would need to broadcast their proposals to all replicas, collect replies, and immediately commit x and y, respectively.

Define scenarios S_j for $j \in \{0, \ldots, n\}$: replicas R_1 through R_j receive the proposal for x before $y (x \rightsquigarrow y)$, while replicas R_{j+1} through R_n have $y \rightsquigarrow x$. An ideal protocol must yield a consistent total order in every scenario S_j.

Case $j = 0$**.** In this trivial case, all replicas have $y \rightsquigarrow x$; the resulting order is $y \rightarrow x$. The symmetric case holds for $j = n$. However, this situation does not always occur. If x and y are proposed concurrently, at least two of the proposing replicas will receive x and y in different orders.

Case $j = 1$**.** All replicas except R_1 have $y \rightsquigarrow x$. At R_n, every replica except R_1 returns the order $y \rightarrow x$, as x has not yet arrived at those replicas, while R_1 returns $x \rightarrow y$. For R_n to commit y immediately within 2δ, it must decide on an order. If R_n decides on $x \rightarrow y$ (committing x before y), but both R_1 and R_n subsequently crash (with R_1 never having broadcast x), the remaining replicas have no record of x and cannot reconstruct the order $x \rightarrow y$, violating consistency. Therefore, to ensure safety, R_n must commit y first, resulting in the order $y \rightarrow x$. A symmetric argument applies for $j = n - 1$, where the order must be $x \rightarrow y$.

Thus, there exists a threshold k such that for $j \leq k$, the order is $y \rightarrow x$, and for $j > k$, the order is $x \rightarrow y$.

Case $j = k$**.** Suppose R_j successfully replies to R_n but not to any other replica. After R_n commits y based on the replies it has collected, both R_j and R_n crash. Since the chosen order between x and y was determined by R_j's reply (unseen by the other replicas), the remaining non-faulty replicas cannot recover the order committed at R_n, again violating consistency.

Since the above cases cover all possible scenarios, the proof is complete. □

This argument does not impose any restrictions on the number of replicas. Under the standard SMR assumption that fewer than half the replicas can fail, when $f = 1, N = 3$ replicas suffice [**?**, 30]. In this special case, a simple protocol can be employed: designate one replica as the leader to determine the order, and have the other two query the leader; commands can then be executed within 2δ.

3 PRIPAXOS: Overview

Recall that PRIPAXOS is designed to achieve low latency under light workload and high throughput under heavy workload (§1). In geo-distributed systems, reducing WAN message delays is crucial for minimizing latency and maintaining performance [4,40]. PRIPAXOS introduces a priority-driven scheduling algorithm to approach near-optimal WAN delays. In addition, PRIPAXOS reduces messaging complexity, enabling sustained high throughput under heavy workloads (§3.1).

A systematic analysis of consensus costs in Theorem1 reveals that neither leader-based nor leaderless approaches achieve optimal efficiency, primarily because both require extra rounds of inter-replica confirmation to finalize command order. In particular, in leader-based protocols, even if a command's position is learned within 2δ, extra message delays are needed to ensure at least f replicas have recorded the order for consistency. In leaderless protocols, additional delays

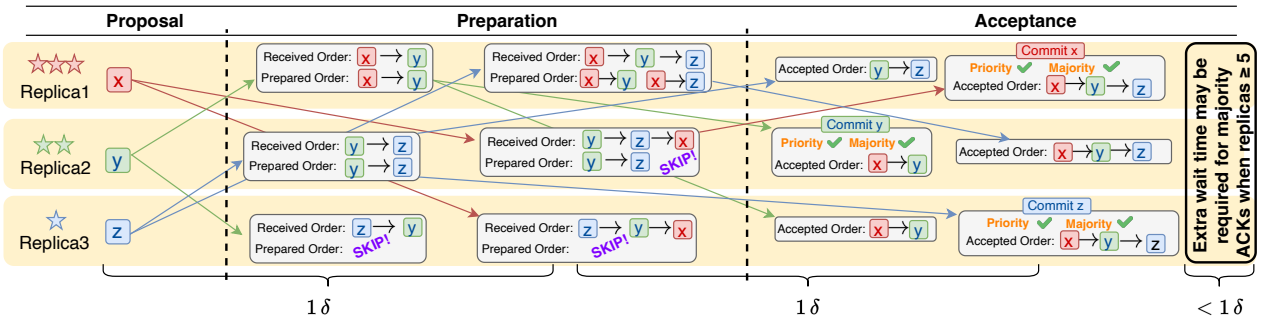

Figure 2: Overview of the PRIPAXOS protocol workflow. The figure illustrates the interactions and message flows among three replicas during command proposal, preparation, and acceptance, highlighting how priority-based ordering enables near-optimal WAN message delays. We present only the messages that are critical for establishing order between replicas. Non-critical messages are omitted for readability.

are also necessary to confirm that a majority-decided order is acknowledged by at least f replicas.

To address this limitation, our primary insight is that if the relative ordering of conflicting commands can be deterministically established during the prepare phase, specifically, as commands are disseminated among replicas, it is possible to obviate the need for an additional round-trip dedicated to ordering decisions. To this end, PRIPAXOS introduces a novel priority-based ordering mechanism: each replica is statically assigned a unique priority. For instance, let R_2 and R_1 be two replicas such that R_2 has higher priority than R_1 (denoted $R_2 \succ R_1$). Suppose R_2 proposes command x while R_1 proposes command y. In the event that x and y conflict, the protocol enforces a deterministic ordering by always placing x before y, denoted as $x \rightarrow y$. This approach enables replicas to independently and consistently resolve conflicts without additional coordination rounds.

To further illustrate its key idea, consider the scenario in Figure 2. Suppose we have three replicas, R_1, R_2, and R_3, deployed across three geographically distinct regions. Assume a static priority assignment such that $R_1 \succ R_2 \succ R_3$. In this example, each replica proposes a distinct, conflicting command: x, y, and z, demonstrating how the protocol deterministically resolves conflicts based on priorities.

When R_1 receives command x from its client, it broadcasts x to the other replicas. As the highest-priority replica, R_1 requires acknowledgments from only a majority of replicas to commit x; with three replicas, a single reply from either R_2 or R_3 suffices. For R_2, any command that originates from a higher-priority replica—such as x from R_1—is deterministically ordered before its own proposed command y. Since commands ordered after y do not influence the commit of y, R_2 only needs a reply from R_1 (and not from R_3) to independently establish a globally consistent order, commit y, and respond to its client. In contrast, R_3 must obtain replies from both R_1 and R_2, ensuring that all higher-priority commands (x and y) are placed before its own command z, thereby enabling R_3 to determine and commit the final order.

Consequently, in each of these scenarios, a single broadcast followed by replies (i.e., 2δ message delays) is sufficient to execute the commands. Notably, R_1, despite being the highest-priority replica, does not assume the role of a traditional leader that enforces a global total order; rather, its responsibility is limited to determining the relative order between its own proposed commands and those it receives from other replicas. Other replicas must, however, receive a reply from R_1 before committing their commands. Importantly, the ordering between y and z is not dictated by their respective arrival times at R_1, but instead is determined solely by the fixed priority order between R_2 and R_3.

An important subtlety arises when the number of replicas exceeds three, and a chain of dependencies is present: in such scenarios, PRIPAXOS may experience a modest increase in execution latency. For example, consider a dependency order $x \rightarrow z$, where R_3 must ensure that x is committed before proceeding to commit z. Upon learning of x from R_1, R_3 must, to guarantee correctness, wait for confirmation from a majority of replicas that x has been received and processed prior to committing z. In the three-replica case illustrated in Figure 2, a single reply from R_1 suffices. However, as the number of replicas increases, the protocol may require additional acknowledgments.

Specifically, with more replicas, R_3 must wait until it learns that R_2 has also received x before committing, thereby shifting the commit trigger from R_1's reply regarding z to R_2's reply concerning x. In the worst case, a full 3δ chain can occur: R_3 broadcasts z to R_1; R_1 broadcasts x to R_2; and R_2 returns the acknowledgment for x to R_3. Notably, because x has already been proposed by R_1 when the message carrying z arrives at R_1, the total delay remains strictly less than 3δ. If no such dependency exists, for instance, if z is non-conflicting across replicas, or R_2 has already broadcast x to R_3 before R_1 replies about z, then 2δ message delays are sufficient.

As demonstrated in §2.2, it is theoretically impossible to achieve a strict 2δ message delay bound for conflicting commands; therefore, the message delays realized by PRIPAXOS are still near-optimal in such scenarios.

3.1 Analysis of Message Complexity

After establishing that consensus can be reached with a minimal number of message delays, PRIPAXOS achieves low

Protocol	Non-conflict Latency	Conflict Latency	Message Complexity (per command)	Quorum Size
Paxos [18]	4δ	4δ	4	$1/2N$
Raft [28]	4δ	4δ	N	$1/2N$
SwiftPaxos [34]	2δ	3δ	$2N$	$1/2N$
EPaxos [25]	2δ	4δ	4	$3/4N$
Mencius [24]	2δ	$3 \sim 4\delta$	3	N
PRIPAXOS	2δ	$2 \sim 3\delta$	$5/8N$	$5/8N$

Table 2: Comparison of latency and message complexity across protocols. For each protocol, we report the number of WAN message delays for non-conflicting and conflicting commands, the message complexity per command at the most heavily loaded replica, and the average quorum size (i.e., the number of replicas contacted per command).

latency under conditions of light client concurrency (❷). In the following, we analyze how the priority mechanism in PRIPAXOS further reduces message complexity to deliver high throughput under heavy workloads (❶).

Bottleneck Analysis. The throughput bottleneck is determined by the replica that receives the greatest number of messages. Consider a system with N replicas, where commands are proposed uniformly across all replicas. The per-command message cost for a given replica consists of two components: the $\frac{1}{N}$ fraction of commands for which it acts as the proposer, and the remaining $\frac{N-1}{N}$ fraction for which another replica is the proposer.

Compared to traditional leader-based protocols, PRIPAXOS does not require a single leader to broadcast and await replies for every command. For commands proposed by other replicas (non-local commands), a replica simply needs to receive broadcast messages. When a replica is the proposer, the per-command message count is $2(N-1)$ (one broadcast plus replies). For commands proposed by other replicas, it receives $N-1$ broadcast messages. The total per-command message cost is therefore:

$$2(N-1)\cdot\frac{1}{N}+(N-1)\cdot\frac{N-1}{N} \;=\; \frac{N^2-1}{N} \;\approx\; N.$$

With further optimizations, this message count can be reduced to approximately $\frac{N}{2}$ per command, and as low as $\frac{5N}{8}$ for the highest-priority replica (see §4.7). In contrast, leader-based protocols such as Raft [28] require the leader to broadcast commands, collect acknowledgments to ensure correctness, and then broadcast commit notifications, resulting in a total communication cost of roughly $3N$ messages per command.

Quorum Size Analysis. The overall message complexity is further influenced by quorum size. If a proposer can obtain responses from a small subset of nearby replicas, the round-trip delay and communication overhead are reduced. In PRIPAXOS, replicas with higher priority require responses from only $(N-1)/2$ peers. As replica priority decreases, the number of required contacts increases, with the lowest-priority replica needing replies from all other replicas, i.e., $(N-1)$ responses. On average, each replica communicates with approximately $\frac{5}{8}N$ peers, which is lower than the requirements of EPaxos and SwiftPaxos under unfixed fast-quorum configurations (approximately $\frac{3}{4}N$; see §3.2). Moreover, since higher-priority replicas contact fewer peers, assigning higher priority to heavily loaded replicas can further enhance the throughput and scalability advantages of PRIPAXOS.

3.2 A Deep Dive into Related Work

Before presenting the detailed design of PRIPAXOS, we analyze the performance characteristics of several state-of-the-art consensus protocols. This comparison serves to highlight the theoretical and practical advances introduced by PRIPAXOS.

Multi-paxos and Raft. In protocols that designate a leader, the leader is responsible for determining the execution order of conflicting commands. For instance, in Raft [28], upon receiving a command from a client, the leader replicates the command to its followers and waits for acknowledgments from a majority of replicas before the command is considered executed. Subsequently, the leader broadcasts acknowledgments (ACKs) to all replicas, resulting in a total delay of 4δ. Since Raft does not incorporate a fast path, this procedure applies equally to both conflicting and non-conflicting commands. The leader sends $3(N-1)$ messages per command: one round for proposal, one for acceptance, and one for ACKs. In practice, the message count can be reduced to N by sending proposals to only half of the replicas and by deferring ACK broadcasts, allowing other replicas to attach ordering results when responding to subsequent commands.

SwiftPaxos. SwiftPaxos [34] introduces a fast quorum mechanism to enable a fast path. With this optimization, a command can be executed within 2δ message delays if the fast quorum reaches agreement on the same order. When conflicting commands are present, the fast quorum adopts the leader's order and returns the result to the client, achieving a latency of 3δ. In terms of message complexity, on the fast path, the majority (i.e., the fast quorum) communicates amongst themselves; on the slow path, messages are broadcast to all replicas to confirm the leader-determined order. Consequently, members of the fast quorum incur $2(N-1)$ messages per command.

Epaxos [25] and its variants [12, 38, 39]. In leaderless protocols such as EPaxos [25] and its descendants [38], commands are executed according to an order determined by a majority of replicas. To ensure correctness, if fewer than $\frac{3}{4}N$ replicas return the same order, the command proposer must broadcast the determined order and await acknowledgments (ACKs) from a majority before executing the command. Moreover, in the

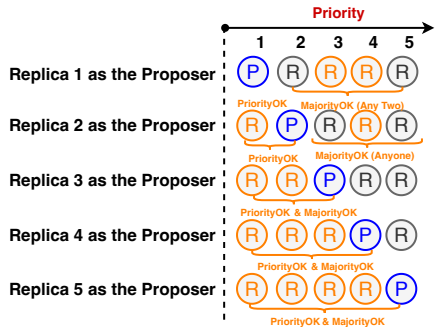

Figure 3: Illustration of the `majorityOKs` and `priorityOKs`.

case of conflicting commands, the command proposer must wait until a complete dependency graph, consistent across all replicas, has been confirmed. Under high contention, this dependency graph may become large, resulting in significant increases in latency. This phenomenon is known as *convoy effect* [7, 38]. For the $1/N$ fraction of commands for which a replica acts as the proposer, EPaxos requires two rounds of broadcast, leading to a latency of 4δ per command.

Mencius. Mencius [24] represents a specialized variant of majority-based consensus protocols. It achieves leaderless operation by rotating the Paxos leader among replicas for successive commands. Command ownership is statically partitioned: replica R is responsible for commands i such that $i \bmod N = R$, and the order of a command is determined at proposal time. However, because any replica may propose a command that must be ordered before previously proposed commands, each replica must collect positive responses from all other replicas before executing command x. As a result, the one-round latency is bounded by the round-trip time (RTT) to the slowest replica. Additionally, to guarantee correctness, the system must verify that all dependencies of x have been committed, leading to a worst-case latency of 4δ.

Comparison and Discussion. Table 2 demonstrates that PRIPAXOS achieves the lowest message delay among all protocols considered to date. For the most common values of N (3 or 5) [8, 9, 22], PRIPAXOS also achieves message counts and contact-set sizes that are nearly optimal.

4 PRIPAXOS Protocols

In this section, we demonstrate how PRIPAXOS leverages replica priorities to efficiently achieve consensus. Specifically, §4.1 outlines the overall protocol workflow, while §4.2 details the priority-based conflict resolution mechanism. Fault tolerance and failover strategies are discussed in §4.3. §4.5 discusses PRIPAXOS's ability to scale and adjust priorities between replicas during runtime. Finally, §4.6 provides a sketch of the protocol's correctness proof.

4.1 Towards Minimal Message Delays

Algorithm 1 outlines the command execution procedure, which consists of the following phases: proposal, preparation, and acceptance.

Proposal. When a replica receives a client request for command x, it designates itself as the proposer for x and assigns it a monotonically increasing instance number. Each replica in PRIPAXOS maintains a local command log that records all observed commands. Upon receiving x, the replica appends it to its log and computes $deps_x$, the set of locally observed commands and their dependencies that conflict with x. Thus, x is considered dependent on these commands and their instances. To track proposal, we use $reach_x$, which records which replicas have learned about x. Whenever a replica receives any message related to x, it marks the sender's entry in $reach_x$ as `true`; the proposer's entry is always `true` by default. The proposer then broadcasts x, along with $deps_x$, to all other replicas.

Algorithm 1: System Workflow of PRIPAXOS

```
when received command(x) from client c for Replica R:
    set R as command proposer P
    add x to command records cmd_P
    for command y in cmd_P do
        if y conflict with x then
            add y and dep_y to dep_x                    // Rule 1
    reach_x[P] = true
    broadcast Propose(x, deps_x, P) to all other replicas
when received Propose(x, deps_x, P) for Replica R:
    add x to cmd_R
    reach_x[P], reach_x[R] = true
    if R ≻ P and R's command y conflict with x then
        if command in deps_y all in deps_x then
            add y to deps_x                             // Rule 2
    broadcast Prepare(x, deps_x, P, R) to all other replicas
when received Prepare(x, deps_x, P, R') for Replica R:
    merge deps_x with local deps_x
    reach_x[R'] = true
    CommitCheck(x, deps_x)
Function CommitCheck(x, deps_x):
    if majority of reach_x = true then
        majorityOKs_x = true
    if all higher priority replicas in reach_x = true then
        priorityOKs_x = true
    if all command in deps_x are committed then
        allCommitted_x = true
    if priorityOKs and allCommitted and majorityOKs then
        CommittedUpTo[replica] ← Prepare.command
        Commit(x)
        if R = P then
            reply to client c
        // check commit condition of y that deps_y = x
        for command y in cmd_R do
            if x in deps_y then
                CommitCheck(y, deps_y)
```

Preparation. Upon receiving command x, a replica appends it to its local log, records its current $deps_x$, and updates $deps_x$ based on the latest state of the log. The replica then broadcasts the updated $deps_x$ and related *acceptance* information to all other replicas.

Acceptance. Upon receiving a *acceptance* request, replica R merges its local $deps_x$ with the $deps_x$ included in the acceptance message, taking, for each replica, the maximum instance number observed. To safely commit x, the replica checks:

- `majorityOk`: a majority of replicas have received x;

Algorithm 2: Recovery Procedure of PRIPAXOS

```
Function Recovery() for Replica R:
    if replica P times out then
        broadcast Remove(P, CommitUpToR(P)) to other replicas
when received Remove(P, CommitUpToR(P)) for Replica R':
    for x ← CommitUpToR(P) + 1 to CommitUpToR'(P) do
        cmdP ← cmdR'[x]
    send Confirm(R', cmdP) to R
when received Confirm(R', cmdP) for Replica R:
    if received Confirm from P then
        return
    for x in cmdP do
        add x to cmdR
        reachx[P], reachx[R] = true, reachx[R'] = true
        merge depsx with local depsx
    if received Confirm from all other replica then
        remove P element from reach, majorityOKs, priorityOKs
        for x in cmdR do
            if x is uncommitted then
                CommitCheck(x, depsx)
```

- `priorityOk`: replies have been received from all higher-priority replicas, ensuring a complete dependency set
- all commands in $deps_x$ have been committed.

The majorityOk and priorityOk conditions are illustrated in Figure 3. If all conditions are satisfied, x is then committed, and the result is returned to the client.

Note that, unlike prior protocols that distinguish between a fast and slow path based on dependency set agreement, PRIPAXOS does not explicitly separate these paths. In PRIPAXOS, whether execution completes within two message delays depends solely on whether all commands in the dependency set have been committed; this is determined by the local commit condition. Therefore, the wait time is typically less than one message delay, contributing to its improved performance.

4.2 Handling Conflicts via Priority Scheduling

We now present PRIPAXOS 's strategy for updating dependency sets to resolve command conflicts. Suppose R_1, R_2, and R_3 propose conflicting commands x, y, and z, respectively, with $R_1 \succ R_2 \succ R_3$. The resolution mechanism is governed by three core rules:

Rule 1 (Real-Time Order): When replica R proposes command c, for all conflicting commands c' already present in its local log there are $c' \rightarrow_r c$ (replica receives c before proposing c'), then we have $c' \rightarrow c$. This rule is transitive and also applies to commands proposed by the same replica.

Rule 1 reflects the command's real-time order and its transitive. This ensures acyclicity: for example, if $x \rightarrow_r y$, and $y \rightarrow_r z$, then $z \rightarrow_r x$ is impossible due to the inherent real-time ordering constraint.

Rule 2 (Priority Scheduling): When command y is received by replica R_1, and $R_1 \succ R_2$, for command x that has been proposed by R_1, we have $x \rightarrow_p y$.

Rule 2 reflects the order based on the priority of replicas. Since priorities among replicas define a partial order, this process is acyclic in isolation.

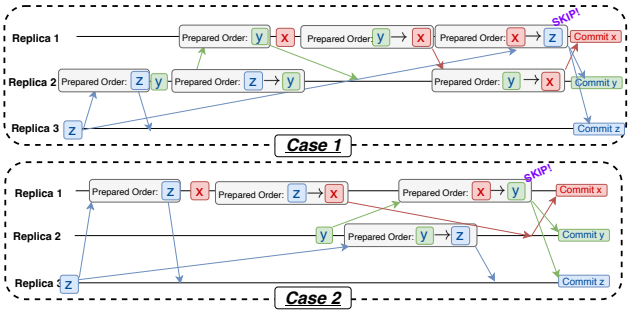

Figure 4: Example of a special dependency scenario in PRIPAXOS.

Rule 3 (Non cycle): *Rule 1* is always applied and *Rule 2* is only applied when it does not introduce a cyclic dependencies.

Applying *Rule 1* and *Rule 2* together can inadvertently create cyclic dependencies. We consider two special cases that *Rule 2* could not be applied as illustarted in Figure 4.

Special case 1 (dependency transitivity): Depends on *Rule 1*, there are $z \rightarrow_r y \rightarrow_r x$, and because R_1 (proposer of x) has higher priority than R_3 (proposer of z), *Rule 2* enforces $x \rightarrow_p z$, introducing a cycle: $z \rightarrow_r y \rightarrow_r x \rightarrow_p z$.

To avoid such a cycle, we apply *Rule 3*. Before setting $x \rightarrow_p z$, we inspect deps_x; although $R_1 \succ R_3$, if deps_x already contains y and y's dependencies include z, then by transitivity $\text{deps}_x = \{z\}$, we do not apply *Rule 2* setting $x \rightarrow_p z$, thereby breaking the potential cycle and yielding a final order of $z \rightarrow y \rightarrow x$.

Special case 2 (dependency prediction): We begin with $z \rightarrow_r x$ by *Rule 1*. As R_1 (proposer of x) has higher priority than R_2 (proposer of y), *Rule 2* implies $x \rightarrow_p y$, and also via *Rule 2*, $y \rightarrow_p z$, potentially forming the cycle $x \rightarrow_p y \rightarrow_p z \rightarrow_r x$. To prevent this, before y adds x, we examine deps_x. Although $R_1 \succ R_2$, if deps_x includes z and $z \notin y$'s dependencies (while $R_2 \succ R_3$), y can infer $\text{deps}_z = y$ and avoid adding x. Then, the final resulting order should be $y \rightarrow z \rightarrow x$.

In summary, since *Rule 1* is always acyclic and *Rule 2* is applied only when it preserves acyclicity, all conflicting commands ultimately form a partially ordered dependency relation, ensuring correctness and consistency in the protocol.

4.3 Priority-driven Recovery Mechanism

Next, we describe how PRIPAXOS recovers from failures both safely and efficiently by leveraging its inherent priority mechanism. When a failed replica is removed from the system, the relative priorities among the remaining replicas remain unchanged, preserving the established ordering guarantees.

Recovering from replica failures. Same as other distributed protocols, PRIPAXOS detects failures via timeouts. When communication with replica R times out, the remaining replicas initiate a consensus process to determine whether R should be removed from the replica quorum. Upon receiving sufficient confirmations, the system ensures that subsequent command commitments no longer require replies from R.

In particular, commands initiated by R continue to be broadcast and committed by all active replicas, as the ordering and

commitment of these commands do not depend on R itself. For commands proposed by other replicas, the primary reason for awaiting a reply from R was to establish the relative ordering with R's commands. Since the final ordering is determined solely by the dependency sets of the involved commands, rather than any private state of R, the replicas now simply wait for all outstanding, uncommitted commands proposed by R to be disseminated and then apply the same dependency resolution rules. Note that, during this process, the `MajorityOk` and `PriorityOk` conditions no longer require a response from R.

As every command must receive acknowledgments from a majority of replicas before being committed, the system maintains sufficient state to recover even in the presence of up to f faulty replicas. From the remaining non-faulty replicas, the system can reconstruct the correct order among committed and outstanding commands, thereby preserving safety.

For commands proposed by R, once they are committed, the result is returned to the client (if still reachable). The client can then select the nearest responsive replica as the proposer for future commands. As long as each client directs each command to a single replica, changing the proposer does not compromise correctness. Importantly, after the failed replica is removed, the relative priority ordering among the remaining replicas remains unchanged.

Recovering from client failures. If a client experiences a timeout and resubmits a command, as the client appends a unique identifier to commands, this allows replicas to detect duplicate submissions and enforce exactly-once execution semantics.

4.4 Heuristic for Priority Assignment

As analyzed in §3.1, PRIPAXOS's performance is sensitive to the assignment of replica priorities. To optimize system efficiency, we introduce a priority allocation strategy specifically tailored to the characteristics of PRIPAXOS. As depicted in Algorithm 1, a replica R's effective priority is principally determined by the set of replicas whose responses fulfill the *priorityOk* and *majorityOk* conditions.

Intuitively, when all replicas experience similar workloads and R is assigned a priority in the upper half of the ordering, it is preferable for the round-trip times (RTTs) between R and all higher-priority replicas to be smaller than the median RTT to any replica. In this scenario, the command latency for R closely approximates the maximum RTT to the nearest majority of replicas, approaching the theoretical minimum. Conversely, if R is positioned in the lower half of the priority ordering, we aim to minimize the largest RTT to any higher-priority replica by assigning the most geographically distant replicas to even lower priorities. As a result, the command latency for R approaches this maximal RTT.

In practice, we can enumerate all possible priority orderings and select the one that minimizes the expected system-wide latency. For each candidate ordering, we estimate the expected latency for every replica as follows: First, gather the RTTs to all higher-priority replicas. If this subset does not yet constitute a majority, we iteratively include the smallest RTTs from the remaining replicas until a majority is reached. The largest RTT in this set is used as the expected latency for that replica. Finally, we compute the average expected latency across all replicas to obtain a global objective value; the ordering that yields the lowest average latency is considered optimal. Algorithm 3 outlines the detailed procedure.

Algorithm 3: Priority Assignment and Re-assignment

```
Function Priority():
    LowestLatency ← ∞
    forEach priority order:
        Latency ← 0
        for R = 1 to N do
            add RTT_{R,1} ... RTT_{R,R-1} to Latencyset
            if R < N/2 then
                add 1/2N − R minimum RTT_{R,i} to Latencyset
            Latency ← Latency + max{LatencySet}
        LowestLatency ← min{LowestLatency, Latency}}
```

When workloads are heterogeneous across replicas, we assign higher priorities to the more heavily loaded replicas. A high-priority replica requires responses from fewer peers, and its commands depend on fewer other commands, resulting in lower latency for its operations. This priority-based strategy effectively reduces the average system latency.

4.5 Scalability and Priority Re-assignment

Next, we discuss how PRIPAXOS can safely scale in and out at runtime. When adding or removing replicas, PRIPAXOS adjusts replica priorities to maintain high performance and reliability, based on its priority heuristic (§4.4).

Scale-In and Scale-Out. PRIPAXOS scales out by simply adding new replicas. A newly joined replica is initially assigned the *lowest* priority. Because commands already in flight on existing replicas do not require replies from the newcomer, they can instead wait for an additional reply from some other replica. For commands proposed by the newly added replica, that replica can observe all other replicas and thus executes exactly the same procedure as Algorithm 1. The safety of all commands is preserved.

Priority Reassignment. After a new replica is added or removed, or when computation/network statistics change at runtime, priorities can be dynamically reordered. We could designate that replica as a temporary coordinator. According to Algorithm 3, when the highest-priority replica determines that the current priority order is suboptimal, it sends a message to other replicas to pause proposing new commands and adopt the new priority order. Because this only prevents new commands from being proposed and does not affect the execution of existing commands, it does not cause correctness issues. When other replicas receive the message, they stop proposing new commands, update their priorities, and send an acknowledgment to the coordinator. After receiving acknowledgments from all replicas, the coordinator notifies them to

resume proposing commands. This mechanism allows the system to restore an efficient priority arrangement when replica latencies fluctuate over extended periods.

4.6 Correctness Proof Sketch

Non-triviality. This follows immediately from the fact that each committed command is sent by clients.

Consistency. Since *Rule 2* creates a dependency between any two conflicting commands unless a dependency already exists, and since the dependency relationship between any two commands is derived solely from the dependency sets carried when commands are emitted from proposers and the priority order among proposers, which is independent of the replica that executes the commands, the execution order of commands is consistent on any replica.

Liveness. A command is committed when its proposer collects replies from a majority of replicas and returns the result to the client. As specified in Algorithm 2, after faulty replicas are removed from the replica set, all remaining non-faulty replicas are able to make progress and commit new commands, thereby guaranteeing liveness.

We now prove that in PRIPAXOS, commands do not form dependency cycles that would prevent progress.

For two commands x and y, consistency ensures that different orders do not form on different replicas. We need to prove that commands do not depend on each other on any replica. Since dependency relationships between commands are derived solely from *Rule 1* and *Rule 2*, we consider the following cases. The order of these cases reflects the temporal order in which dependencies are established on a replica.

Case 1: $x \rightarrow_r y \rightarrow_r x$. This does not hold because *Rule 1* ensures the dependencies are acyclic.

Case 2: $x \rightarrow_p y \rightarrow_r x$. By *Rule 1*, replica r has already received y before x is proposed. Therefore, it is impossible to record the dependency $x \rightarrow_p y$ before this point.

Case 3: $x \rightarrow_r y \rightarrow_p x$ or $x \rightarrow_p y \rightarrow_p x$. Since *Rule 2* checks whether x exists in $deps_y$ before execution, it is impossible to record the dependency $y \rightarrow_p x$ after $x \rightarrow_p y$.

We now consider cases involving cycles of three or more commands. For commands proposed by the same replica, we can conceptually split the replica into multiple adjacent priority replicas, where earlier commands are proposed by higher-priority replicas and arrive at their corresponding lower-priority replicas before later commands are proposed. Therefore, to simplify the presentation, we use replicas to represent commands proposed in it. Since there exists an order between any two replicas, this forms a directed complete graph, i.e., a tournament.

It is well-known that the shortest directed cycle in a tournament (if one exists) has length 3. Therefore, when a cycle exists among replicas, assume that R_1, R_2, and R_3 form a cycle. The possible orders are $R_1 \rightarrow R_3 \rightarrow R_2 \rightarrow R_1$ and $R_1 \rightarrow R_2 \rightarrow R_3 \rightarrow R_1$.

Case 1: $R_1 \rightarrow R_3 \rightarrow R_2 \rightarrow R_1$. Since *Rule 2* is a partial order and only creates dependency order from higher-priority replicas to lower-priority replicas, if a cycle exists, it must be due to *Rule 1* creating an dependency order from a lower-priority replica to a higher-priority replica. Since *Rule 1* cannot form cycles, there must be $R_1 \rightarrow_r R_3 \rightarrow_r R_2 \rightarrow_p R_1$, which is exactly the situation described in *Special Case 1* in §4.2. Therefore the dependency $R_2 \rightarrow R_1$ will not be formed in this case.

Case 2: $R_1 \rightarrow R_2 \rightarrow R_3 \rightarrow R_1$. As in the same condition as *Case 1*, there must be $R_3 \rightarrow_r R_1$. If the other two dependency orders are derived from *Rule 2*, this is the situation described in *Special Case 2*. If one of the other two dependency orders is derived from *Rule 1*, for example $R_1 \rightarrow_r R_2$, similar to *Special Case 1*, since *Rule 2* checks the transitivity of dependencies before execution, the dependency $R_2 \rightarrow_p R_3$ will not be formed.

Our discussion is exhaustive, which proves that in PRIPAXOS commands do not form dependency cycles.

4.7 Implementation-level Optimizations

Thrifty Communication. Similar to the optimizations in EPaxos, PRIPAXOS can apply thrifty communication strategies to further minimize message complexity. Higher-priority replicas, in addition to proposing commands to other high-priority peers, broadcast only to a geographically proximate majority, thus obtaining sufficient replies without incurring the overhead of network-wide broadcasts. This approach can be extended: instead of broadcasting REPLY messages to all replicas, it suffices to guarantee that every replica receives a majority of replies, thereby maintaining safety. This optimization can reduce the number of messages sent by the highest-priority replica to approximately $\frac{5}{8}N$.

Efficient Metadata Encoding. In addition to reducing message counts, PRIPAXOS minimizes network load by keeping message metadata compact. Following the approach in EPaxos, PRIPAXOS tracks only the highest instance number of each conflicting replica, rather than including the full set of conflicting instances. Because dependencies are transitive, this per-replica tracking suffices to prevent premature commits of commands with unresolved conflicts. Consequently, PRIPAXOS maintains lightweight messages whose size scales linearly with the number of replicas.

5 Evaluation

5.1 Evaluation Setup

Our evaluation benchmarks PRIPAXOS against five state-of-the-art consensus protocols: Paxos [18], Fastpaxos [19] (functionally similar to Raft), EPaxos [25], SwiftPaxos [34], and Mencius [24]. All protocols are implemented in Go, leveraging a common SwiftPaxos [34] codebase to ensure comparability, and the source code is publicly available.

Experiments are conducted on Amazon EC2, deploying five replicas across distinct geographic regions (Taipei (AP), UAE (ME), Tel Aviv (IL), N. California (US), and São

	AP	ME	IL	US	SA
AP	—				
ME	139	—			
IL	142	244	—		
US	252	137	194	—	
SA	303	289	245	173	—

Table 3: The network RTT (in ms) measured between regions.

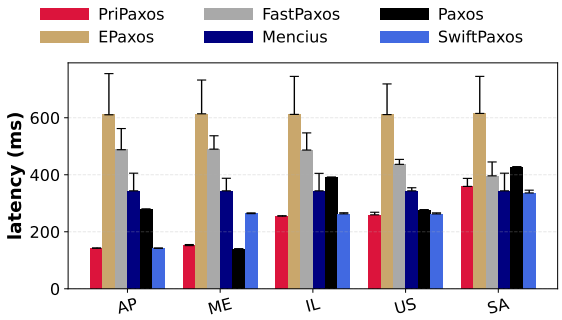

Figure 5: Median latency at each of the five geo-distributed replicas. The 90th-percentile latency is indicated by lines above the bars. Replicas are presented in the order of PRIPAXOS's assigned priority.

Paulo (SA)), each on a separate continent to capture a wide range of network latencies. Clients, co-located within each region, issue requests concurrently and record per-command execution latency. All nodes run on Amazon Linux 2 virtual machines equipped with 16 vCPUs and 32 GB RAM. Clients operate in a closed-loop mode, issuing a new command only after the previous completes.

Benchmark Configuration. We evaluate each protocol using a standard transactional workload comprising three operations: *insert*, *get*, and *update*, with each record uniquely identified by a key. Command conflicts arise when two operations access the same key and at least one is a write, modeling typical contention patterns in real-world applications. To simulate varying conflict rates, each client issues commands targeting a key with a specified probability or selects a unique key otherwise, following established benchmarking practice. All commands carry a 1 KB payload (default as YCSB). The default conflict rate is set to 80%, following the 80-20 principle and approximating the default skew in YCSB when Zipf is enabled.

To ensure comparability across protocols, we assign AP (the replica with the lowest average network latency to others) as the leader for all leader-based protocols. For PRIPAXOS, priorities are determined according to the methodology described in §4.4, resulting in the order: AP, ME, IL, US, and SA. In SwiftPaxos, the fast quorum is configured to include the higher-priority nodes ME and IL.

5.2 Latency in Wide-Area Networks

In this experiment, we demonstrate that PRIPAXOS consistently achieves the lowest average latency among all protocols tested in wide-area network deployments. We deploy 100 clients per region to generate sufficient traffic without overloading the replicas. Figure 5 presents both the mean and

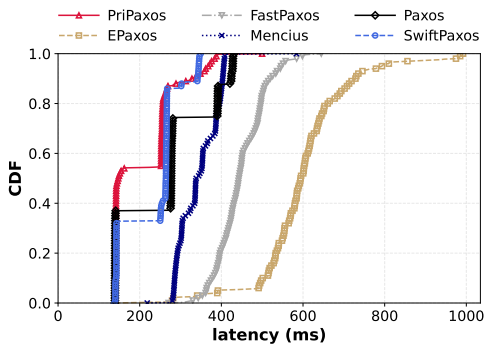

Figure 6: Cumulative distribution function (CDF) of execution latency across all replicas in the 80% conflict scenario.

90th-percentile latencies for each protocol.

In PRIPAXOS, high-priority replicas (e.g., AP, ME) are required to wait for acknowledgments from fewer peers, enabling them to attain significantly lower latencies relative to other protocols. For instance, the replica in region AP only needs replies from ME and IL (see Table 3), resulting in a low measured latency of approximately 142 ms. Conversely, the lowest-priority replica (SA) must obtain responses from all other replicas. In scenarios where this involves communicating with the most distant replica, the round-trip time can approach three additional one-way WAN delays, yielding an average latency of 359 ms. This is marginally higher than SwiftPaxos (336 ms) and Mencius (343 ms).

SwiftPaxos leverages a fast path to execute command within two or three message delays, delivering low latency for most replicas. Since region AP is the leader of SwiftPaxos, commands proposed by region AP only need to receive replies from two regions to execute, resulting in lower latency. However, for replicas in other regions, it is often necessary to wait for responses from the full fast quorum ME and IL, potentially incurring up to three message delays. As a result, the latency in the region ME is notably higher than PRIPAXOS (263 ms versus 150 ms), and in the region IL and US, SwiftPaxos also exhibits slightly higher latency compared to PRIPAXOS.

For Fastpaxos, the probability of achieving the fast path is extremely low in practice, since it is rare for replicas to receive conflicting or concurrent commands in the same order. Thus, Fastpaxos primarily executes commands via slow path and incurs poor latency performance.

Under 80% conflict rates, EPaxos must expand its dependency graph to resolve contention among commands, which substantially increases both the mean and 90th-percentile latencies across all regions.

5.3 Tail latency

Figure 6 shows the latency distribution for PRIPAXOS corresponding to the scenario in Figure 5. The median (50th-percentile) latency for PRIPAXOS is approximately 145 ms, which is higher than that of the other protocols. This is due to PRIPAXOS performs well in three regions (AP, ME and IL).

For commands proposed by the lowest-priority replica (SA), it must wait for acknowledgments from all other replicas before committing. As a result, tail latency for PRIPAXOS exhibits moderate variability, with the 90th-percentile latency reaching around 327 ms. Despite this, PRIPAXOS achieves lower tail latency than all protocols except SwiftPaxos, demonstrating its robustness under high contention.

Leader-based protocols such as SwiftPaxos and Paxos display relatively stable tail latencies, as the leader's role centralizes dependency coordination. In contrast, EPaxos experiences significant tail latency degradation, especially under high conflict rates due to the *convoy effect*.

5.4 Impact of the Conflict Rate

To assess the effect of contention on protocol performance, we vary the conflict rate from 0% to 100% in 10% increments. Figure 7 presents the latency averaged across all sites.

PRIPAXOS does not introduce additional communication overhead under contention; instead, it simply waits for the resolution of necessary dependencies before safely committing commands. As a result, its latency remains nearly constant across all conflict rates, ranging from 202 ms to 206 ms.

For EPaxos, when the conflict rate is 0%, each replica only needs to communicate with its two nearest neighbors, allowing commands to be executed within two message delays and achieving a minimum latency of 176 ms. However, as the conflict rate increases, the size and complexity of the dependency graph in EPaxos grow, leading to a sharp increase in waiting times and mean latency, which peaks at ∼600 ms.

To reason about optimal performance, we estimate a *theoretical optimal latency* of 171 ms based on EPaxos's performance at 0% conflict—this value represents the minimum possible delay if every replica could commit within two message delays by contacting its two closest peers. As discussed in §2.2, this lower bound is unattainable in the presence of conflicting commands. At a conflict rate of 100%, PRIPAXOS achieves a latency of 203 ms, outperforming the current state-of-the-art SwiftPaxos, which reaches 235 ms. PRIPAXOS's latency is approximately 15.3% away from *theoretical optimal latency*, compared to SwiftPaxos which is 37.4% away from *theoretical optimal latency*, representing an improvement of approximately 244%.

Protocols without a fast path such as Paxos and Mencius, exhibit flat latency profiles across all conflict rates, with latencies of 260 ms and 345 ms, respectively. When contention arises, SwiftPaxos reverts to its slow path, necessitating extra communication to maintain a consistent order, which causes its latency to increase modestly from 225 ms at 0% conflict rate to 235 ms at 100% conflict rate.

5.5 Throughput Comparison

To evaluate the throughput of PRIPAXOS under heavy workloads, we conduct an experiment that incrementally increases the system load to saturation. We vary the total number of clients from 100 to 3000, as shown in Figure 8 and Figure 9.

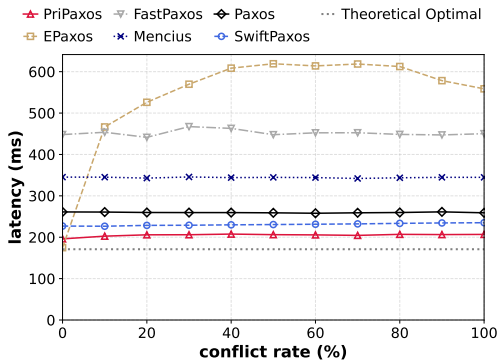

Figure 7: Average latency as the conflict rate increases from 0% to 100%. Each data point represents the mean latency across all replicas, highlighting the protocols' sensitivity to contention.

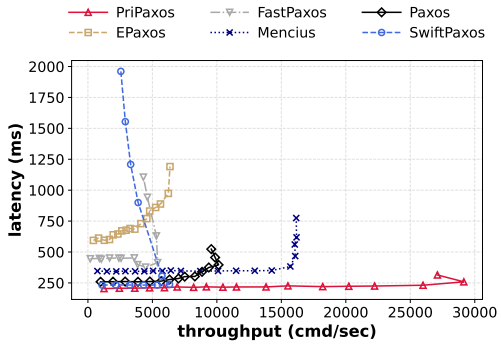

Figure 8: Throughput and latency as the number of clients increases from 100 to 3000 under an 80% conflict rate with 5 replicas.

As client concurrency grows, the throughput bottleneck transitions to the message-processing capacity of the leader replica. In PRIPAXOS, with 5 replicas, the highest-priority replica's average message number is $\frac{5}{8} \times 5 = \frac{25}{8}$ per command, it is fewer than any other protocol except Mencius's 3 messages. Combined with PRIPAXOS's lower latency, this communication efficiency enables it to achieve significantly higher throughput than competing protocols, peaking at $\sim 29,000\ cmd/s$. Since PRIPAXOS does not execute commands via fast path, the leader's processing rate remains stable even as contention increases, resulting in throughput that is largely unaffected by rising conflict rates.

Protocols without a fast path, such as Mencius, also exhibit relatively constant throughput as conflict rates increase. However, since the average latency of Mencius is approximately twice that of PRIPAXOS, its maximum throughput is about half as much ($\sim 16,000\ cmd/s$). For SwiftPaxos, the proportion of slow-path commands rises with increasing conflict, and its leader's message number overhead is the highest among all protocols evaluated (i.e., 10 messages per command with 5 replicas), causing its throughput to decline sharply.

5.6 Service Availability under Failures

We analyze the effect of failures in both high-priority replicas (AP, acting as leader) and low-priority replicas (ME, non-leaders) on system availability and throughput. Figure 10

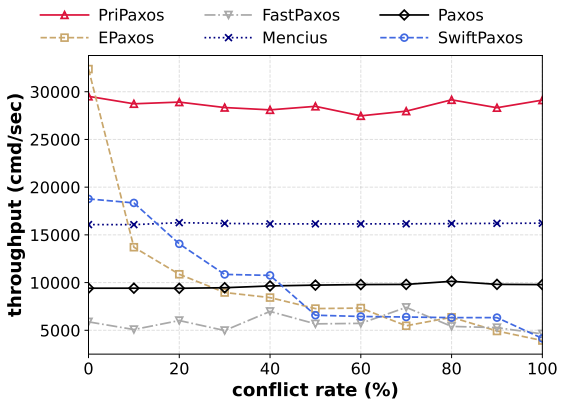

Figure 9: System throughput as a function of conflict rate, ranging from 0% to 100%. Higher conflict rates generally reduce throughput due to increased coordination and dependency resolution overhead.

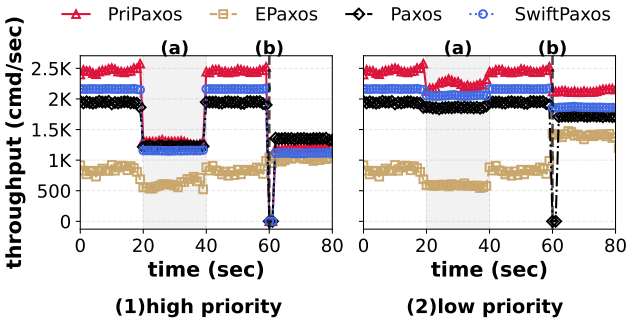

Figure 10: System throughput over time under fault scenarios: (a) slowdown of either a high-priority (leader) or low-priority (non-leader) replica, and (b) failure of the leader.

illustrates the response of PRIPAXOS, SwiftPaxos, EPaxos, and Paxos to asynchrony and replica failures. In the first scenario, after 20 s, the network latency at one site is artificially increased by 200 ms and subsequently restored to normal after another 20 s (interval (a)). This is followed by a simulated replica crash (event (b)).

High-priority Replica Slowdowns. When the high-priority replica and its co-located client experience increased latency, both PRIPAXOS and SwiftPaxos exhibit a pronounced decrease in throughput. This is expected, as both protocols require every command to be acknowledged by the high-priority replica (or leader) before commitment. In contrast, as EPaxos and Paxos are leaderless protocols, they show only a modest reduction in throughput.

High-priority Replica Crashes. If the high-priority replica fails, throughput in PRIPAXOS, SwiftPaxos, and Paxos drops to zero, since these protocols cannot make progress without the leader or high-priority replica. EPaxos continues to function at a reduced capacity, as its decentralized architecture allows it to proceed as long as a majority is available. As the number of clients decreases, the throughput of PRIPAXOS, SwiftPaxos, and Paxos decreases. Interestingly, EPaxos benefits from a simplified dependency graph when fewer replicas are present, which leads to increased throughput when a replica fails.

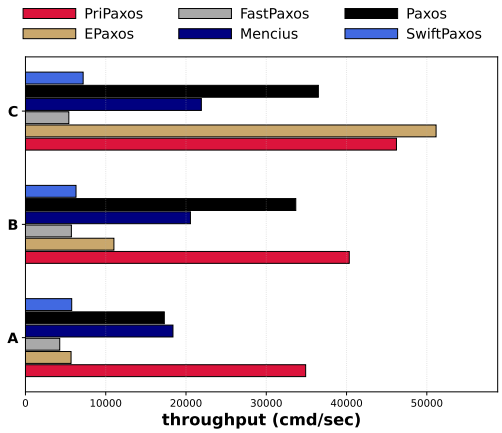

Figure 11: Throughput under different YCSB workloads.

Low-priority Replicas Slowdowns or Crashes. The system's behavior is similar to the high-priority case, but with a crucial distinction: when a low-priority replica slows down, commands can still be committed without waiting for its response. As a result, protocols experience only a minor reduction in throughput. Furthermore, if a low-priority replica crashes, these protocols except Paxos degrade gracefully by slowing down rather than halting entirely.

5.7 YCSB Workload

In the final experiment, we evaluate PRIPAXOS using the standard YCSB benchmark, which comprises workloads with a Zipfian distribution (0.8 skew). Workload A is update-heavy (50% updates, 50% gets), workload B is read-heavy (5% updates, 95% gets), and workload C is read-only. The results are shown in Figure 11.

In workload A, the high proportion of updates (50%) results in greater data transmission per command, leading to lower throughput across all protocols. Consistent with the results in Figure 9, PRIPAXOS achieves the highest throughput at 35,000 cmd/s, approximately twice the throughput of Mencius which reaches 18,000 cmd/s. Due to frequent command conflicts, EPaxos exhibits relatively poor performance under this workload.

By contrast, EPaxos has highest throughput in the read-only workload C. As in the scenario shown in Figure 9 at a 0% conflict rate, all commands in this workload execute in two message delays.

6 Conclusion

We present PRIPAXOS, a new Paxos-based state machine replication protocol. It executes commands in two message delays without contention and in fewer than three delays otherwise. We show that its latency is near-optimal in theory, and experiments demonstrate up to 13.6% lower latency and 4.4× higher throughput than state-of-the-art alternatives SwiftPaxos.

References

[1] ZooKeeper. https://zookeeper.apache.org/.

[2] Yahoo! Cloud Serving Benchmark. https://github.com/brianfrankcooper/YCSB, 2004.

[3] Ailidani Ailijiang, Aleksey Charapko, Murat Demirbas, and Tevfik Kosar. Wpaxos: Wide area network flexible consensus. *IEEE Transactions on Parallel and Distributed Systems*, 31(1):211–223, 2019.

[4] Masoud Saeida Ardekani and Douglas B Terry. A {Self-Configurable}{Geo-Replicated} cloud storage system. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 367–381, 2014.

[5] Jason Baker, Chris Bond, James C Corbett, JJ Furman, Andrey Khorlin, James Larson, Jean-Michel Leon, Yawei Li, Alexander Lloyd, and Vadim Yushprakh. Megastore: Providing scalable, highly available storage for interactive services. In *CIDR*, volume 11, pages 223–234, 2011.

[6] Carlos Eduardo Bezerra, Fernando Pedone, and Robbert Van Renesse. Scalable state-machine replication. In *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 331–342. IEEE, 2014.

[7] Matthew Burke, Audrey Cheng, and Wyatt Lloyd. Gryff: Unifying consensus and shared registers. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 591–617, 2020.

[8] Yu Lin Chen, Shuai Mu, Jinyang Li, Cheng Huang, Jin Li, Aaron Ogus, and Douglas Phillips. Giza: Erasure coding objects across global data centers. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 539–551, 2017.

[9] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google's globally-distributed database. In *Proceedings of the Tenth Symposium on Operating Systems Design and Implementation (OSDI '12)*, October 2012.

[10] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's highly available key-value store. *ACM SIGOPS operating systems review*, 41(6):205–220, 2007.

[11] Vitor Enes, Carlos Baquero, Alexey Gotsman, and Pierre Sutra. Efficient replication via timestamp stability. In *Proceedings of the Sixteenth European Conference on Computer Systems*, pages 178–193, 2021.

[12] Vitor Enes, Carlos Baquero, Tuanir França Rezende, Alexey Gotsman, Matthieu Perrin, and Pierre Sutra. State-machine replication for planet-scale systems. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–15, 2020.

[13] Tuanir França Rezende and Pierre Sutra. Leaderless state-machine replication: Specification, properties, limits. In *34th International Symposium on Distributed Computing (DISC 2020)*, pages 24–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2020.

[14] Maurice P Herlihy and Jeannette M Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 12(3):463–492, 1990.

[15] Flavio P Junqueira, Benjamin C Reed, and Marco Serafini. Zab: High-performance broadcast for primary-backup systems. In *2011 IEEE/IFIP 41st International Conference on Dependable Systems & Networks (DSN)*, pages 245–256. IEEE, 2011.

[16] Tim Kraska, Gene Pang, Michael J Franklin, Samuel Madden, and Alan Fekete. Mdcc: Multi-data center consistency. In *Proceedings of the 8th ACM European Conference on Computer Systems*, pages 113–126, 2013.

[17] Jay Kreps, Neha Narkhede, Jun Rao, et al. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB*, volume 11, pages 1–7. Athens, Greece, 2011.

[18] Leslie Lamport. The part-time parliament. *ACM Trans. Comput. Syst.*, 16(2):133–169, 1998.

[19] Leslie Lamport. Fast paxos. *Distributed Computing*, 19(2):79–103, 2006.

[20] Leslie Lamport. Lower bounds for asynchronous consensus. *Distributed Computing*, 19(2):104–125, 2006.

[21] Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Vertical paxos and primary-backup replication. In *Proceedings of the 28th ACM symposium on Principles of distributed computing*, pages 312–313, 2009.

[22] Leslie Lamport and Mike Massa. Cheap paxos. In *International Conference on Dependable Systems and Networks, 2004*, pages 307–314. IEEE, 2004.

[23] Jialin Li, Ellis Michael, Naveen Kr Sharma, Adriana Szekeres, and Dan RK Ports. Just say {NO} to paxos overhead: Replacing consensus with network ordering. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 467–483, 2016.

[24] Yanhua Mao, Flavio Paiva Junqueira, and Keith Marzullo. Mencius: building efficient replicated state machines for wans. In *Proceedings of the 8th USENIX conference on Operating systems design and implementation*, volume 8, pages 369–384, 2008.

[25] Iulian Moraru, David G. Andersen, and Michael Kaminsky. There is more consensus in egalitarian parliaments. In *Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP '91)*, November 2013.

[26] Brian M Oki and Barbara H Liskov. Viewstamped replication: A new primary copy method to support highly-available distributed systems. In *Proceedings of the seventh annual ACM Symposium on Principles of distributed computing*, pages 8–17, 1988.

[27] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX annual technical conference (USENIX ATC 14)*, pages 305–319, 2014.

[28] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the USENIX Annual Technical Conference (ATC '14)*, June 2014.

[29] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, et al. Zanzibar:{Google's} consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 33–46, 2019.

[30] Seo Jin Park and John Ousterhout. Exploiting commutativity for practical fast replication. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 47–64, 2019.

[31] Fernando Pedone and André Schiper. Generic broadcast. In *International symposium on distributed computing*, pages 94–106. Springer, 1999.

[32] Sebastiano Peluso, Alexandru Turcu, Roberto Palmieri, Giuliano Losa, and Binoy Ravindran. Making fast consensus generally faster. In *2016 46th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 156–167. IEEE, 2016.

[33] Tuanir F Rezende, Pierre Sutra, Rodrigo Q Saramago, and Lasaro Camargos. On making generalized paxos practical. In *2017 IEEE 31st International Conference on Advanced Information Networking and Applications (AINA)*, pages 347–354. IEEE, 2017.

[34] Fedor Ryabinin, Alexey Gotsman, and Pierre Sutra. {SwiftPaxos}: Fast {Geo-Replicated} state machines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 345–369, 2024.

[35] Fred B Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *Acm Computing Surveys (CSUR)*, 22(4):299–319, 1990.

[36] Pierre Sutra and Marc Shapiro. Fast genuine generalized consensus. In *2011 IEEE 30th International Symposium on Reliable Distributed Systems*, pages 255–264. IEEE, 2011.

[37] Rebecca Taft, Irfan Sharif, Andrei Matei, Nathan VanBenschoten, Jordan Lewis, Tobias Grieger, Kai Niemi, Andy Woods, Anne Birzin, Raphael Poss, et al. Cockroachdb: The resilient geo-distributed sql database. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1493–1509, 2020.

[38] Sarah Tollman, Seo Jin Park, and John Ousterhout. {EPaxos} revisited. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 613–632, 2021.

[39] Alexandru Turcu, Sebastiano Peluso, Roberto Palmieri, and Binoy Ravindran. Be general and don't give up consistency in geo-replicated transactional systems. In *International Conference on Principles of Distributed Systems*, pages 33–48. Springer, 2014.

[40] Bekir Turkkan, Elvis Rodrigues, Tevfik Kosar, Aleksey Charapko, Ailidani Ailijiang, and Murat Demirbas. How to evaluate distributed coordination systems?–a survey and analysis. *arXiv preprint arXiv:2403.09445*, 2024.

[41] William E Weihl. Commutativity-based concurrency control for abstract data types. *IEEE transactions on computers*, 37(12):1488–1505, 2002.

[42] Michael Whittaker, Neil Giridharan, Adriana Szekeres, Joseph M Hellerstein, and Ion Stoica. Bipartisan paxos: A modular state machine replication protocol. *arXiv preprint arXiv:2003.00331*, 2020.

[43] Jonathan Zarnstorff, Lucas Lebow, Christopher Siems, Dillon Remuck, Colin Ruiz, and Lewis Tseng. Racos: Improving erasure coding state machine replication using leaderless consensus. In *Proceedings of the 2024*

ACM Symposium on Cloud Computing, pages 600–617, 2024.

[44] Irene Zhang, Naveen Kr. Sharma, Adriana Szekeres, Arvind Krishnamurthy, and Dan R. K. Ports. Building Consistent Transactions with Inconsistent Replication. *ACM Trans. Comput. Syst.*, 35(4):1–37, December 2018.

[45] Hanyu Zhao, Quanlu Zhang, Zhi Yang, Ming Wu, and Yafei Dai. Sdpaxos: Building efficient semi-decentralized geo-replicated state machines. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 68–81, 2018.